

# FOOD SURPLUS EXCHANGE

## Backend Implementation Blueprint — Hackathon MVP

A two-sided surplus-food marketplace: businesses list food that is still safe to eat but at risk of waste; nearby consumers discover, reserve/pay, and collect it within a defined pickup window.

### Product principle

Seller creates surplus listing → consumer discovers → reserves → seller sees reservation → consumer collects → order completes → inventory and impact update.

## 1. What We Are Building

The product connects restaurants, bakeries, cafes, hostels and grocery stores with nearby consumers who can buy surplus food at a reduced price before it becomes waste.

**Seller:** create listing → quantity/price/pickup window → publish

**Consumer:** discover → view → reserve/buy → pickup

**System:** orders + inventory + notifications + impact

TABETE's official flow similarly covers nearby discovery, payment, a pickup time and store pickup; its owner side uses a web management interface for listings.

■cite■turn0search0■turn0search3■

## 2. Users & Interfaces

| Role     | Interface     | Main actions                                                                                  |
|----------|---------------|-----------------------------------------------------------------------------------------------|
| Consumer | Web app       | Register/login, location, browse, filter, view listing, reserve/buy, order history, pickup.   |
| Seller   | Web dashboard | Register store, create/edit/cancel listing, view reservations, confirm handover, view impact. |
| Admin    | Simple admin  | Approve sellers, moderate listings, monitor orders and metrics.                               |

## 3. Backend Architecture

Recommended hackathon stack: **React/Vite** → **REST API** → **Node.js/Express** → **MongoDB Atlas**. Keep it a modular monolith; microservices are unnecessary for a 3-day MVP.

```
Frontend
↓ HTTPS/REST
Express API
■ ■ Auth / Users
■ ■ Stores
■ ■ Listings
■ ■ Discovery + Geo
■ ■ Orders + Reservations
■ ■ Notifications
■ ■ Impact
■ ■ Admin
↓
MongoDB Atlas
```

## 4. MongoDB Data Model

| Collection    | Purpose                | Key fields                                                                                                             |
|---------------|------------------------|------------------------------------------------------------------------------------------------------------------------|
| users         | Identity/roles         | _id, name, email, phone, role, passwordHash, location                                                                  |
| stores        | Seller business        | _id, ownerId, name, address, geo, openingHours, status                                                                 |
| listings      | Surplus inventory      | _id, storeId, title, category, quantity, remainingQty, originalPrice, rescuePrice, pickupStart, pickupEnd, status, geo |
| orders        | Reservation/purchase   | _id, userId, listingId, storeId, quantity, amount, status, pickupCode, createdAt                                       |
| payments      | Payment state          | _id, orderId, provider, amount, status, transactionRef                                                                 |
| notifications | Event messages         | _id, userId, type, payload, read, createdAt                                                                            |
| impact        | Rescued-food metrics   | _id, orderId, quantityRescued, estimatedWasteAvoided                                                                   |
| reports       | Seller/admin analytics | _id, storeId, period, sales, rescuedQty, wasteAvoided                                                                  |

## 5. Listing & Order State

Listing: DRAFT → ACTIVE → PARTIALLY\_RESERVED → SOLD\_OUT / EXPIRED; ■ CANCELLED

Order: PENDING → RESERVED → PAID → READY\_FOR\_PICKUP → COMPLETED; ■ CANCELLED / EXPIRED / REFUNDED

- Only ACTIVE listings with stock and a valid pickup window are visible.
- Reservation atomically reduces remainingQty.
- When stock reaches zero, listing becomes SOLD\_OUT.
- After pickupEnd, unresolved listings become EXPIRED.

- Only the owning seller can edit/cancel a listing.
- Only an eligible order can be completed.

## 6. REST API

| Method | Endpoint                  | Purpose          |
|--------|---------------------------|------------------|
| POST   | /api/auth/register        | Create account   |
| POST   | /api/auth/login           | Authenticate     |
| GET    | /api/auth/me              | Current user     |
| GET    | /api/listings             | Browse/filter    |
| GET    | /api/listings/:id         | Details          |
| GET    | /api/listings/nearby      | Nearby discovery |
| POST   | /api/orders               | Reserve/order    |
| GET    | /api/orders               | Order history    |
| GET    | /api/orders/:id           | Order details    |
| POST   | /api/orders/:id/cancel    | Cancel           |
| POST   | /api/stores               | Create store     |
| GET    | /api/stores/me            | Seller store     |
| POST   | /api/stores/:id/listings  | Create listing   |
| PATCH  | /api/listings/:id         | Edit listing     |
| POST   | /api/listings/:id/publish | Publish          |
| POST   | /api/listings/:id/cancel  | Cancel listing   |
| GET    | /api/stores/:id/orders    | Seller orders    |
| POST   | /api/orders/:id/complete  | Confirm pickup   |
| GET    | /api/stores/:id/analytics | Seller metrics   |

## 7. Authentication & Authorization

- JWT access tokens for the MVP.
- bcrypt/argon2 password hashing; never store plain passwords.
- Roles: CONSUMER, SELLER, ADMIN.
- Seller may modify only their own store/listings.
- Consumer may access only their own private orders.
- Admin endpoints require ADMIN role.

## 8. Nearby Discovery

Store documents use a GeoJSON Point + 2dsphere index. Nearby API filters ACTIVE listings, remainingQty > 0 and pickupEnd > now, then sorts by distance and/or urgency.

```
GET /api/listings/nearby?lat=15.x&lng;=74.x&radius;=5000
```

## 9. Reservation — Critical Backend Logic

### Do not trust frontend stock checks.

Two consumers may reserve simultaneously. The backend must perform the availability check and quantity decrement atomically. Example: remainingQty=3; A requests 2 and B requests 2 → only one reservation succeeds.

This is one of the most important backend pieces because it demonstrates real transaction/concurrency handling.

## 10. Payment

- Use mock payment or a provider's test mode for the 3-day MVP.
- Keep payment status separate from order status.
- Never store card numbers/CVV in MongoDB.
- If a provider is used, verify payment server-side.

TABETE's live service uses payment before pickup; our MVP can reproduce the business sequence with a test/mock payment path. ■cite■turn0search3■

## 11. Pickup Verification

- Generate a short pickup code/QR for confirmed orders.

- Seller enters/scans the code.
- Backend verifies store ownership and pickup eligibility.
- Order becomes COMPLETED.
- Inventory and impact metrics update.

## 12. Notifications

| Event             | Recipient        | Purpose               |
|-------------------|------------------|-----------------------|
| LISTING_PUBLISHED | Nearby consumers | New surplus available |
| ORDER_CREATED     | Seller           | New reservation       |
| PAYMENT_CONFIRMED | Consumer/Seller  | Order confirmed       |
| PICKUP_REMINDER   | Consumer         | Pickup approaching    |
| ORDER_COMPLETED   | Consumer         | Rescue completed      |
| LISTING_EXPIRING  | Seller           | Listing ending soon   |

## 13. Impact & Analytics

- Food portions/items rescued.
- Completed rescue orders.
- Estimated value of food rescued.
- Seller surplus revenue.
- Active stores and consumers.
- Estimated waste avoided — explicitly labelled as an estimate.

TABETE positions the model around reducing food waste while helping stores reduce waste and generate sales, so these metrics fit the product purpose.

## 14. Optional AI Layer

AI should enhance the marketplace, not be a dependency. If the core flow is stable, add one focused feature:

- Surplus-demand prediction from historical listings/orders.
- Rescue-price suggestion based on pickup time, stock and sell-through.
- Smart ranking based on distance, urgency, price and user preferences.

**Priority:** rule-based ranking first; ML only if time remains.

## 15. Backend Folder Structure

```
backend/
├── src/
│   ├── config/ (db.js, env.js)
│   ├── models/ (User, Store, Listing, Order, Payment, Notification)
│   ├── controllers/
│   ├── routes/
│   ├── middleware/ (auth, roles, errorHandler)
│   ├── services/ (orderService, listingService, notificationService, impactService)
│   ├── app.js
│   ├── tests/
│   ├── .env
│   ├── package.json
│   └── README.md
```

## 16. Step-by-Step Build Order

1. Lock MVP: Consumer + Seller + basic Admin; defer complex features.
2. Initialize Express, CORS, error handler and /health.
3. Create MongoDB Atlas database and indexes.
4. Implement User, registration, hashing, login and JWT.
5. Implement Store and seller ownership.
6. Implement Listing with quantity, pricing, GeoJSON and pickup window.
7. Build seller listing APIs: create, edit, publish, cancel.
8. Build consumer discovery: nearby, filters, sorting and details.
9. Build reservation/order logic with atomic stock updates.
10. Add mock/test payment state.
11. Build pickup code/QR verification and completion.
12. Add notifications.
13. Add impact metrics and seller analytics.
14. Add basic admin endpoints.
15. Connect frontend and test endpoints.
16. Test double reservation, expiry, cancellation, unauthorized access and duplicate pickup.
17. Deploy API and configure environment variables.
18. Run the Golden Demo repeatedly.

## 17. Three-Day Execution Plan

| Time      | Backend goal             | Deliverable                   |
|-----------|--------------------------|-------------------------------|
| Day 1 AM  | Architecture + DB + auth | API + MongoDB + login         |
| Day 1 PM  | Stores + listings        | Seller publishes surplus      |
| Day 1 Eve | Discovery                | Consumer sees nearby listings |
| Day 2 AM  | Orders                   | Safe reservation/quantity     |
| Day 2 PM  | Pickup + notifications   | Complete handover             |
| Day 2 Eve | Impact + admin           | Metrics + monitoring          |
| Day 3 AM  | Integration/testing      | Full flow                     |
| Day 3 PM  | Deploy/polish            | Live demo                     |
| Final     | Rehearse                 | Reliable Golden Demo          |

## 18. Golden Demo

1. Seller logs in.
2. Seller creates '10 Fresh Sandwiches — \$60 — pickup 7–8 PM'.
3. Listing becomes ACTIVE.
4. Consumer sees it nearby.
5. Consumer reserves 2.
6. Backend validates stock and creates order.

7. Seller sees reservation.
8. Consumer receives pickup code.
9. Seller verifies code.
10. Order becomes COMPLETED.
11. Remaining quantity updates.
12. Impact metrics update.
13. Judge sees MongoDB data and the transaction relationship.

#### Demo rule

Demo one complete transaction from seller → consumer → pickup → measurable impact. Do not demo disconnected pages.

## 19. TABETE — Reference Model

TABETE connects users with surplus food from bakeries, restaurants, hotels, supermarkets and other businesses. Its user flow includes nearby discovery, payment, pickup time and store pickup; its owner FAQ describes web management and on-demand listing.

■cite■turn0search0■turn0search3■

| TABETE pattern           | Our MVP                         |
|--------------------------|---------------------------------|
| Nearby surplus discovery | Geo-based listing discovery     |
| Seller listing           | Seller dashboard + listing APIs |
| Pickup window            | pickupStart/pickupEnd           |
| Payment                  | Test/mock payment first         |
| Pickup confirmation      | Pickup code/QR                  |
| Web store management     | Seller web dashboard            |
| Waste reduction          | Impact metrics                  |

## 20. Backend Definition of Done

- MongoDB Atlas connected
- Auth + roles work
- Seller can create/publish listing
- Consumer can discover nearby listing
- Consumer can reserve
- Overselling prevented
- Seller sees order
- Pickup code works
- Seller can complete pickup
- Inventory/status updates
- Order history works
- Notifications work
- Impact metrics update
- API deployed
- Golden Demo works repeatedly

## 21. Start Here

**Do not code 30 endpoints first.** Lock the four core entities and one transaction:

User → Store → Listing → Order

**Build priority:** Auth → Store → Listing → Discovery → Reservation → Pickup → Impact → Notifications → Payment/Al.